

Annex III — Proof of Authoritative Capability (PoAC™)

Governed Evolution of the WR Graph™

Document status: Normative specification. Version 1.1 (see §9). This Annex is self-contained except for two declared dependencies: the normative structure of the WR Graph™ is defined in Annex II §II.2.4, and the mutation classifications assigned in Annex II (§§II.4, II.5, II.6) are instances of the taxonomy defined herein. It carries no other cross-references.

Applies to: WR Desk™ runtime, WR Graph™, Workflow Context Recorder (WCR), BEAP™-adjacent learning operations, acquisition of externally produced artifacts, provisioning of graph content to paired Field Devices, and all optimization processes executed during idle periods ("Night Pipeline").

Trademark notice: Optirando™, WR Desk™, WR Graph™, BEAP™, PoAE™, PoAC™, and WR Code® are used throughout as product and protocol designations of the Optirando™ system.

§1 Purpose and Scope

§1.1 Motivation

The WR Graph™ is defined normatively in Annex II §II.2.4 as the unifying context layer of the system, combining four knowledge dimensions: declarative (facts, documents, retrieval sources), episodic (execution history, feedback events, outcomes), procedural (skills, WCR recordings, workflows, prompt profiles), and parametric (adapters and other trained artifacts).

Annex II specifies what the WR Graph™ contains. This Annex specifies how the WR Graph™ is permitted to change. It is the dynamic complement to the static definition: no dimension of the WR Graph™ holds a right of mutation without demonstrable authority. The machinery specified in this Annex constitutes the **graph governance process** referenced by Annex II §II.2.4; background sorting, prioritization, classification, and self-optimization of the four dimensions are operations of this process and are classified under §4.

§1.2 Core Principle

Proof of Authoritative Capability (PoAC™) extends the trust model of Proof of Authoritative Execution (PoAE™) from the execution boundary to the acquisition boundary:

Where PoAE™ guarantees that no action is executed without an authorized, auditable path, PoAC™ guarantees that no capability enters the system — no skill, no WCR recording, no knowledge source, no prompt profile, no validator, no adapter, whether produced locally or acquired externally — and no graph state is mutated, without an authorized, auditable path.

PoAE™ proves that a capability was exercised with authority; PoAC™ proves that the capability itself was acquired with authority. Together they close the loop: everything the system can do, and everything it does, is traceable to a demonstrable grant.

The system's optimization objective is Long-Term Local Capability Growth: project knowledge and automation capability are improved iteratively across all four knowledge dimensions, using idle compute time, on local hardware, under local control.

§1.3 What PoAC™ Governs

PoAC™ governs every mutation of the WR Graph™, including but not limited to:

1. Ingestion of new declarative sources (RAG corpus changes)
2. Creation and revision of procedural entities (skills, WCR recordings, prompt profiles, validators, workflows)
3. Creation, training, activation, and retirement of parametric artifacts (adapters, specialized models)
4. Derivation and condensation of episodic knowledge (feedback events, outcome aggregation)
5. Routing and confidence metadata affecting any of the above
6. Any interaction with cloud services in service of learning
7. Acquisition and admission of externally produced artifacts (adapters, templates, recordings, validators) obtained through any distribution channel
8. Provisioning of graph content to paired Field Devices, and its recall or expiry (Annex II §II.6)

§1.4 What PoAC™ Does Not Govern

PoAC™ does not govern task execution at runtime (governed by PoAE™), transport security (governed by the sealed transport specification), vault semantics (governed by the vault specification), or the transport and packaging mechanics of distribution channels (governed by the Canon and Annex II); PoAC™ governs the consent, evidence, and provenance obligations of acquisition and provisioning events, not their wire-level mechanics. PoAC™ does not impose any legal interpretation on the operator; it records provenance and enforces operator-defined policy (see §3.6).

§2 Definitions

Capability. In this Annex, a capability is any WR Graph™ entity that contributes to the system's ability to act or answer, across all four knowledge dimensions: declarative sources and their index segments, episodic condensations, procedural entities (skills, WCR recordings, workflows, prompt profiles, validators, tools), parametric artifacts (adapters, specialized models), and the routing metadata binding them. The term is deliberately broad: a RAG source is a capability in the same sense as a trained adapter — both extend what the system can do, and both require an authorized acquisition path. *This sense is distinct from the Capability Token of the Canon's PoAE™ model, which denotes an execution-authorization scope. A capability in the PoAC™ sense is something the system knows or can do; a Capability Token is a permission to act. Nothing in this Annex creates, expands, or modifies Capability Tokens.*

Derived Capability. A capability produced locally by the deployment: authored, recorded, elicited, ingested from operator-provided sources, or trained from local data sets. Its provenance is a derivation chain (§3.1).

Acquired Capability. A capability produced outside the deployment and obtained through a distribution channel: a manufacturer adapter, a published template, an externally authored recording or validator. Its provenance is an acquisition evidence record (§3.8) in lieu of a derivation chain.

Distribution Channel. Any mechanism by which externally produced artifacts enter or leave the deployment — repository, peer-to-peer transfer, manufacturer delivery, or gateway forward channel. Channel mechanics are outside this Annex; the acquisition and publication events they carry are governed here.

Provisioning. The delivery of sealed, version-stamped, lease-bound graph content to a paired Field Device under Annex II §II.6.

Mutation. Any create, update, invalidate, or delete operation on a WR Graph™ entity or edge.

Evidence Event. A persisted, timestamped record of an observable signal from daytime operation: a user feedback action, a run outcome, a retrieval-quality measurement, or a correction.

Gap. A typed, evidence-backed hypothesis that the system's current knowledge or capability is insufficient for a recurring class of work. A gap without linked evidence is invalid and **MUST NOT** be surfaced.

Proposal. A concrete, reviewable remediation for a gap or an improvement opportunity, produced by the Night Pipeline and presented in the Morning Digest.

Consent Record. A persisted, timestamped record of an operator decision (accept / reject / defer) on a proposal, bound to the proposal's identity and content hash.

Morning Digest. The bounded set of proposals presented to the operator after a Night Pipeline run.

Night Pipeline. The ordered sequence of idle-time optimization stages defined in §6.

Provenance Chain. The complete derivation or acquisition record of a graph entity: its originating evidence, its consent record(s), its source(s) or acquisition evidence, and its downstream dependents.

§3 Normative Area 1 — Cross-Dimensional Provenance Model

§3.1 Universal Provenance Requirement

Every entity in the WR Graph™, in every knowledge dimension, **MUST** be able to answer three questions at any time:

1. **Origin** — From which evidence event(s), source(s), and consent record(s) does this entity derive?
2. **Derivation** — Which upstream entities does it build upon (data sets, sources, prior versions)?
3. **Dependency** — Which downstream entities depend on it (the deletion cascade)?

For a Derived Capability, the Origin and Derivation questions are answered by its derivation chain. For an Acquired Capability, they are answered by its acquisition evidence record (§3.8): the acquisition event is the origin, and the attestation chain — publisher identity, signature, hash, version — stands in for the derivation, whose internal composition remains with the publisher. An acquired entity with complete acquisition evidence is **NOT** provenance-orphaned.

An entity for which any of the three questions is unanswerable under the applicable model is **provenance-orphaned** and MUST be flagged in the next Morning Digest.

§3.2 Rationale (Informative)

Provenance is required for reasons independent of any legal consideration:

- **Regenerability.** Parametric artifacts are bound to a base model. Base models change. For derived artifacts, the durable asset is the data set and the training recipe, not the weights; regeneration on a new base model requires the full derivation chain. For acquired artifacts, the durable reference is the acquisition record: it identifies the publisher and version from which a replacement compatible with a new base model can be sought.
- **Diagnosis.** When an artifact underperforms, the first diagnostic question is which source introduced the defect — or, for acquired artifacts, which publisher and version.
- **Deletion cascade.** When a source is withdrawn (contract end, operator decision, data-subject request, publisher revocation), every derived entity must be identifiable and re-derivable or retirable.
- **Auditability.** Organizational deployment requires that any capability of the system can be traced to an authorized acquisition path.

§3.3 Source Entities

Every external source (document, book, manual, API documentation, web resource, purchased material, cloud-generated content) MUST be represented as a Source entity with at minimum:

Field	Requirement
source_id	Stable identifier
acquisition	How obtained (user-provided / discovered / purchased / cloud-generated / WCR-derived)
acquired_at	Timestamp
consent_ref	Consent record authorizing ingestion
license_status	cleared / unclear / restricted — as declared by the operator
training_permitted	yes / no / unclear — as declared by the operator
content_hash	Integrity binding

The system records these fields; it does not adjudicate them. Declaration is an operator act.

§3.4 Data Set Entities

Training data sets are first-class graph entities. A data set MUST reference all Source entities it draws from, the generation method (extraction / synthesis / paraphrase / WCR-derivation), the generating component (local model / cloud service, with the cloud abstraction boundary of §7 applying), and quality metrics from its evaluation runs.

For Derived Capabilities, data sets, not adapters, are the persistent asset. Derived adapters are treated as compiled caches of data sets against a specific base model and are regenerable by design. Acquired adapters are not regenerable by the deployment; their lifecycle follows §3.8 and §6.3.

§3.5 Deletion Cascade

When a Source entity is invalidated, the system MUST identify all dependent data sets, adapters, prompt profiles, and RAG index segments, and MUST present a cascade plan (retire / regenerate without the source / retain with flag) as a Class-3 proposal (§4). Silent retention of derivatives of an invalidated source is prohibited.

Invalidation of an acquisition record — publisher revocation, withdrawal by the distribution channel, or failed re-verification of signature or hash — SHALL trigger the same cascade treatment for the acquired artifact and all entities depending on it, with the cascade options being retire / replace via the original channel / retain with flag.

§3.6 Policy Gate (Configurable)

Whether entities with `license_status: unclear` or `training_permitted: unclear` may enter the training path is a policy configuration, not a fixed system rule:

- **Default (single-operator deployments):** open — provenance is recorded, no gate is enforced.
- **Organizational policy:** a deployment MAY enforce that only cleared/yes sources enter data-set generation and training.

The policy setting itself, and every change to it, is a logged graph mutation.

§3.7 Temporal Validity

All provenance-bearing edges carry bi-temporal validity (`valid_from`, `valid_until`, `recorded_at`). Contradicting knowledge invalidates prior edges rather than deleting them. Confidence values decay in the absence of confirming evidence; decay parameters are deployment configuration.

§3.8 Acquisition Provenance for Externally Produced Artifacts

Every Acquired Capability MUST be represented with an acquisition evidence record containing at minimum:

Field	Requirement
<code>artifact_id</code>	Stable identifier
<code>publisher</code>	Verified publisher identity (verification per the publisher-verification requirements of Annex II §II.3.2)
<code>signature</code>	Publisher signature over the artifact
<code>content_hash</code>	Integrity binding, verified before admission and re-verifiable at any time
<code>version</code>	Publisher-declared version
<code>declared_scope</code>	The competence, domain, or context regions the artifact claims to serve
<code>base_model_binding</code>	For parametric artifacts: the base model identity and version the artifact is bound to
<code>channel</code>	Distribution channel and origin (repository identity, peer identity, gateway H-ID)
<code>acquired_at</code>	Timestamp
<code>consent_ref</code>	Consent Record authorizing acquisition and admission (Class 3, §4)
<code>usage_terms</code>	License or usage terms as declared by the publisher and acknowledged by the

Field

Requirement

operator

Verification of signature and content hash against this record is a precondition of admission (Annex II §II.2.6) and **MUST** be repeatable on demand. An acquired artifact whose re-verification fails **SHALL** be treated as an invalidated acquisition under §3.5.

The system records and verifies these fields; it does not adjudicate usage terms. Acknowledgement is an operator act, consistent with §3.3.

§4 Normative Area 2 — Mutation Class Taxonomy

§4.1 Principle

Learning is continuous and fine-grained; consent is a scarce operator resource. Requiring consent for every mutation would render the system inoperable; requiring consent for none would render it ungoverned. PoAC™ therefore partitions all mutations into three classes. **Every mutation operation implemented in the system MUST be assigned to exactly one class, and the assignment table (§4.5) is normative.**

§4.2 Class 1 — Autonomous Hygiene

Mutations that are purely derived, reconstructible from existing data, and do not introduce new knowledge or capability.

- Executed autonomously during the Night Pipeline.
- Logged (append-only), not surfaced in the Morning Digest except in aggregate ("hygiene summary").
- **MUST** be reversible by recomputation.

Examples: confidence updates and decay, embedding re-indexing over already-ingested content, linking evidence events to existing entities, episodic condensation that preserves the underlying events, statistics aggregation, automatic classification of an admitted entity into its topic region.

§4.3 Class 2 — Reported Adjustments

Mutations that influence system behavior but are trivially reversible and carry no acquisition of new knowledge, no new capability, and no external contact beyond already-consented relationships.

- Executed autonomously, reported item-by-item in the Morning Digest with a one-action rollback.
- **MUST** retain the prior state for rollback for a configurable retention window.

Examples: routing weight adjustments among existing implementations, retrieval parameter tuning within an existing corpus, re-ranking of existing skills for an existing context, activation-threshold adjustments, supersession or deprecation of an automation path with preserved lineage, generation of prepared packages from admitted content, provisioning of non-confidential admitted content to a paired Field Device (reversible by recall or expiry).

A deployment **MAY**, by policy, promote any Class-2 operation to Class 3.

§4.4 Class 3 — Consent-Gated Mutations

Mutations that introduce new knowledge or new capability into the WR Graph™, alter procedural behavior, involve any external contact in service of learning, publish any artifact to an external party, or change the exposure of confidential content.

- MUST NOT be applied without an explicit Consent Record.
- Presented as proposals in the Morning Digest, each carrying its evidence chain (§5), its provenance plan (§3), and where applicable a before/after demonstration.
- Rejected proposals are persisted with their rejection; rejection dampens future proposals of the same type (§5.6).

Examples: ingestion of any new source; creation or revision of prompt profiles, validators, WCR recordings, or workflows; data-set generation; adapter training, activation, or retirement; acquisition and admission of any externally produced artifact; publication of any artifact through a distribution channel; deletion cascades; any cloud interaction in service of learning; provisioning of confidential content, credential references, or delegated scopes to a Field Device; Field Device pairing or unpairing; policy changes.

No Class-3 mutation is ever applied silently. This mirrors, and is normatively identical in spirit to, the WCR self-optimization rule: propose, never apply silently.

§4.5 Assignment Table (Normative)

Operation	Class
Confidence update / decay	1
Embedding re-index (existing corpus)	1
Evidence-event linking	1
Episodic condensation (events preserved)	1
Recurring-action pattern detection (opt-in, §5.7)	1
Automatic classification of an admitted entity into a topic region	1
Lease renewal of an existing grant or provision (includes revocation and version check)	1
Routing weight adjustment	2
Retrieval parameter tuning (existing corpus)	2
Skill re-ranking for context	2
Supersession / deprecation of an automation path (lineage preserved)	2
Prepared-package generation from admitted content	2
Provisioning of non-confidential admitted content to a paired Field Device	2
New RAG source ingestion	3
Prompt profile creation / revision	3
Validator creation / revision	3
WCR recording revision	3
Data-set generation	3
Adapter training / activation / retirement	3
Acquisition and admission of an externally produced artifact	3
Publication of an artifact through a distribution channel	3
Deletion cascade execution	3

Operation	Class
Any learning-related cloud interaction	3
Provisioning of confidential content, credential references, or delegated scopes to a Field Device	3
Field Device pairing / unpairing	3
Policy configuration change	3
Implementations MUST extend this table before introducing any new mutation operation; an unclassified mutation operation is a specification violation.	

§5 Normative Area 3 — Evidence, Feedback Events, and Gap Typology

§5.1 Evidence Requirement

A gap is valid only if it is backed by linked evidence events. The Night Pipeline MUST NOT surface any gap or proposal that cannot present its evidence. "Topic occurred" is not evidence of a gap; "topic recurred and performance was demonstrably insufficient" is.

§5.2 Feedback Event Schema

The primary evidence signal is explicit operator feedback. A Feedback Event MUST persist:

Field	Content
event_id	Stable identifier
target_ref	The output, run, or WCR execution being rated
skill_ref	The procedural entity that produced the target (if any)
verdict	positive / negative
commentary	Operator free text (optional but strongly weighted)
project_ref	Project / domain context
timestamp	Event time

Secondary evidence signals (run outcomes, retrieval-quality scores, reformulation detection, correction diffs) follow the same event pattern and MAY be added incrementally; their absence does not block V1 operation (§6.6).

§5.3 Baseline Check

Before a knowledge gap may be declared, the Night Pipeline MUST verify that the deficiency is real: the affected queries of the day are re-evaluated against the base model plus the existing corpus. A gap is declared only where the baseline demonstrably fails. Commodity knowledge already mastered by the base model MUST NOT generate proposals.

§5.4 Gap Typology (Normative)

Every Gap entity carries exactly one type; the type determines the remediation path and its cost class:

Gap type	Meaning	Remediation path (ascending cost)
Knowledge gap	Required knowledge exists	Source discovery → RAG ingestion → prompt

Gap type	Meaning	Remediation path (ascending cost)
(external)	publicly but not in the corpus	profile → (last) data set + adapter
Knowledge gap (internal)	Required knowledge exists only with the operator/organization	Elicitation: the system requests the knowledge from the operator (dictation, document, WCR recording)
Capability gap	Knowledge suffices; a tool or renderer is missing	Tool integration → workflow
Process gap	A recurring procedure has no recorded skill	WCR recording proposal → workflow compilation
Quality gap	Outputs are produced but repeatedly corrected	Validator → prompt profile revision → (last) adapter

Where a configured distribution channel offers a suitable, verifiable externally produced artifact, **external acquisition** (§3.8, Class 3) MAY be proposed as a remediation rung for any gap type, positioned by cost before adapter training (§6.2).

§5.5 Internal Knowledge Elicitation

The internal knowledge gap is a first-class mechanism, not a fallback. Where evidence shows recurring reference to undocumented internal matter (processes, decisions, conventions), the Night Pipeline SHALL generate an elicitation proposal ("Process Y is referenced repeatedly but described nowhere — record it?"). Elicited knowledge enters the graph as a Source with **acquisition: user-provided** and full provenance.

§5.6 Digest Budget and Meta-Feedback

- The Morning Digest is hard-bounded (default: five proposals), ranked by expected utility × evidence strength.
- Every proposal outcome (accepted / rejected / ignored) is itself an evidence event on the proposing mechanism. Rejected proposal types MUST dampen future ranking of that type. The operator trains the proposer.

§5.7 Recurring-Action Detection (Opt-In)

Where enabled by configuration, the system detects recurring action patterns in episodic evidence — sequences of manual operations, queries, or executions that repeat across sessions within the same semantic context — and treats them as candidate process gaps (§5.4).

- Detection itself operates exclusively on locally recorded episodic events and is a Class-1 operation (derived, reconstructible, no new knowledge).
- Where a pattern exceeds a configurable recurrence threshold, the Night Pipeline SHALL generate a WCR recording proposal for the Morning Digest ("This sequence has recurred N times in project Z — record it as a reusable automation?"), carrying the recurrence evidence as its evidence chain.
- The proposal is Class 3: no recording session is initiated, and no procedural entity is created, without an explicit Consent Record.
- An accepted proposal leads to an operator-performed WCR recording; the resulting skill enters the graph with **acquisition: user-provided** provenance and is routed per §6.4.

- The feature is opt-in: with detection disabled, no recurrence analysis over episodic events is performed.

§6 Normative Area 4 — Night Pipeline and Escalation Ladder

§6.1 Pipeline Stages (Ordered)

The Night Pipeline executes in fixed order; each stage is independently useful and independently interruptible:

1. **Graph hygiene** (Class 1): confidence decay, re-indexing, event linking, episodic condensation, automatic classification of admitted entities.
2. **Baseline re-evaluation** (Class 1): the day's affected queries evaluated against base model + existing corpus; parametric artifacts A/B-checked against the bare base model on real queries of the day.
3. **Gap analysis** (produces Class-3 proposals): typed gaps with evidence chains, including recurring-action patterns where detection is enabled (§5.7), ranked into the Digest budget.
4. **Remediation drafting** (produces Class-3 proposals): concrete artifacts — revised prompt profiles counter-evaluated against the objected examples ("would the revision have addressed the critique?"), source-discovery results, validator drafts, elicitation requests, and, where a distribution channel is configured, acquisition candidates with their acquisition evidence (§3.8).
5. **Skill maintenance and provisioning upkeep** (Classes 1–3 as applicable): WCR endpoint indexing, precondition edges (§6.4), drift detection, re-verification of acquisition evidence for acquired artifacts (§3.8), lease renewals (Class 1), and generation or refresh of prepared packages for provisioned devices (Class 2), including packages assembled for operator-declared upcoming contexts (Annex II §II.6.5).
6. **Data-set generation and training** (Class 3, consent required per step): executed only for accepted gaps that the preceding stages demonstrably could not close, and only from Sources admitted by the active policy gate (§3.6). Training uses the locally configured fine-tuning stack; resulting adapters enter the graph with full provenance and telemetry hooks (§6.3).

§6.2 Escalation Ladder

Remediation cost ascends; the Pipeline **MUST** exhaust cheaper rungs before proposing more expensive ones:

Workflow compilation → prompt profile → RAG enrichment → validator → tool integration → external acquisition (where a channel is configured) → adapter training.

Adapter training is justified only where (a) a gap is persistent across baseline checks, (b) cheaper rungs — including external acquisition where available — have been tried or ruled out with recorded rationale, and (c) sufficient admissible data exists. The expected steady state is that most gaps close below the adapter rung.

§6.3 Parametric Telemetry and Retirement

Every active adapter — derived or acquired — MUST accumulate per-adapter telemetry: acceptance rate versus correction rate, and periodic idle-time A/B comparison against the bare base model. An adapter that no longer outperforms its base model — including after a base-model upgrade — MUST be proposed for retirement (Class 3).

Retirement semantics differ by provenance class:

- **Derived adapters:** retired adapters remain in the graph as provenance records; their data sets remain available for regeneration on a new base model.
- **Acquired adapters:** retirement means deactivation with the acquisition record retained as provenance. The deployment SHALL NOT attempt regeneration (no data set is held). Where the original channel remains configured, the retirement proposal MAY include an update check for a publisher version compatible with the current base model; installing such a version is a new acquisition (Class 3, §3.8).

§6.4 Skill Routing (Two-Stage)

Procedural skills (including WCR endpoints stored directly in the graph) are routed in two stages:

1. **Semantic stage:** embedding retrieval over skill descriptions yields candidates for the current context.
2. **Precondition stage:** graph traversal filters candidates by recorded preconditions — target system reachability, project context, UI/version compatibility, authorization.

Below a confidence threshold, a matching skill is offered, not executed; above it, it may enter the standard execution preview. Routing is a graph property (edges: skill → precondition, skill → project, skill → confidence), not a separate subsystem.

§6.5 Temporal Complement to Intent Validation (Informative)

Daytime intent validation asks whether the question posed is the right question. The Night Pipeline asks the temporal complement: whether the knowledge exists to pose the right question at all. Gap analysis is intent validation extended into the time dimension.

§6.6 Minimal Viable Loop (V1)

A conforming minimal implementation consists of: the Feedback Event schema (§5.2), Class-3 proposal generation restricted to prompt profiles and RAG suggestions, the Digest with budget and meta-feedback (§5.6), and Class-1 hygiene. Baseline checks, elicitation, source discovery, telemetry, acquisition, and training are conforming extensions in that order. No stage of V1 requires training, cloud access, a configured distribution channel, or any license consideration.

§7 Normative Area 5 — Cloud Abstraction Boundary (Learning-Assist)

§7.1 Principle and Delimitation

Cloud services MAY assist learning; they MUST NOT receive operational knowledge. The boundary is defined as a data-class allowlist, not as a list of permitted functions, because functions leak differently depending on prompt content.

This boundary governs **learning-assist interactions with cloud services** exclusively. It does not govern the publication or acquisition of artifacts through distribution channels: those are separate, consent-gated acts (Class 3, §4.5) whose mechanics are defined outside this Annex under the Distribution Boundary. Publication of an artifact through a distribution channel is not a crossing of this boundary; conversely, no learning-assist cloud interaction may be reframed as distribution to escape the prohibitions of §7.3.

§7.2 Permitted Outbound Data Classes

Only the following MAY cross the boundary, and only under a Class-3 consent for the interaction:

1. **Taxonomy-level skill descriptors** — generic subject identifiers ("OAuth token refresh", "invoice validation", "polynomial simplification") with no instance data.
2. **Generic generation requests** — requests for synthetic examples, benchmark templates, or curriculum structures phrased exclusively at taxonomy level.
3. **Public-literature queries** — searches over public material.

§7.3 Prohibited Outbound Data Classes

The following MUST NOT cross this boundary under any configuration:

- WR Graph™ structure, content, or excerpts, in any dimension
- WCR recordings or fragments thereof
- Evidence events, feedback commentary, or correction content
- Usage frequencies and recurrence statistics (metadata leaks business activity)
- Internal source content or identifiers
- Any confidential material as classified by the local classifier

§7.4 Local Instantiation Step

Cloud-generated artifacts are generic by construction. Mapping generic artifacts onto concrete projects — real field names, real processes, real documents — is a **mandatory local step** performed by local models only. Cloud output enters the graph as a Source with **acquisition: cloud-generated** and is subject to the same provenance and policy rules as any other source.

§7.5 Local-Only Guarantee

Every learning function specified in this Annex MUST be operable with cloud access disabled entirely and with no distribution channel configured. Cloud assistance and external acquisition are accelerants, never dependencies. Degradation to local-only mode is a configuration act, not an architectural change.

§8 Relation to Prior Work (Informative)

This Annex composes known mechanisms under a governance model; the individual mechanisms are not claimed as novel.

- **Idle-time optimization:** sleep-time compute approaches (Letta, 2025) established that agent systems benefit from offline reflection between interactions. PoAC™ differs in placing the entire offline process under consent, evidence, and provenance obligations.
- **Skill accumulation:** skill libraries in embodied and workflow agents (Voyager, Wang et al., 2023; Agent Workflow Memory, 2024) demonstrated persistent capability growth. PoAC™ adds precondition-gated routing and provenance-bound lifecycle management.
- **Offline prompt optimization:** programmatic prompt optimization (DSPy / MIPROv2) is the pattern instantiated at the prompt-profile rung of the escalation ladder.
- **Adapter libraries and routing:** modular LoRA libraries with learned routing (PHATGOOSE and successors) are the nearest relatives of the parametric dimension; PoAC™ treats derived adapters as regenerable caches of provenance-bound data sets rather than as primary assets, and binds acquired adapters to attestation-based acquisition provenance.
- **Package management and software supply chains:** signature- and hash-verified acquisition of externally produced artifacts (§3.8) instantiates established software supply-chain practice (signed packages, verified publishers, reproducible integrity checks); PoAC™ extends it with consent-gated admission and graph-integrated dependency cascades.
- **Active learning:** evidence-driven gap detection descends from active-learning practice; the internal-knowledge elicitation mechanism (§5.5) inverts conventional retrieval augmentation by identifying absent knowledge and requesting it from its only holder.

The claimed contribution is the normative composition: a knowledge graph in a governed system holds no right of mutation — in any of its dimensions — without demonstrable, auditable authority. The acquisition of every capability, whether derived locally or acquired externally, is subject to the same proof discipline as its execution.

§9 Version History

Version	Date	Change
1.0	2026-07-07	Initial normative specification under the designation PoAC™ (Proof of Authoritative Capability): capability definition, cross-dimensional provenance model, mutation class taxonomy, evidence and gap typology including opt-in recurring-action detection, night pipeline and escalation ladder, cloud abstraction boundary.
1.1	2026-07-09	WR Graph™ definition dependency re-pointed to Annex II §II.2.4; self-containment statement corrected; graph governance process identification (§1.1). Scope extended to external acquisition and Field Device provisioning (§1.3 items 7–8; §1.4 delimitation). Definitions added: Derived/Acquired Capability, Distribution Channel, Provisioning; Capability Token disambiguation (§2). §3.1 provenance requirement extended to attestation-based acquisition evidence; §3.4 regenerability restricted to derived adapters; §3.5 cascade extended to invalidated acquisitions; new §3.8 Acquisition Provenance schema. §4.5 assignment table extended: auto-classification and lease renewal (Class 1); supersession, prepared-package generation, non-confidential Field Device provisioning (Class 2); external

Version Date**Change**

acquisition/admission, publication, confidential provisioning, device pairing/unpairing (Class 3); §4.3/§4.4 class definitions aligned. §5.4 external-acquisition remediation rung; §6.1 stages extended (acquisition candidates, re-verification, lease renewals); §6.2 ladder gains external-acquisition rung before adapter training; §6.3 retirement semantics split by provenance class. §7 retitled Learning-Assist and delimited from the Distribution Boundary; §7.5 local-only guarantee extended to no-channel operation; §6.6 V1 unaffected. §8 supply-chain prior-work entry added. §6.1 stage 5 extended with prepared-package generation and refresh, including operator-declared Forward Preparation per Annex II §II.6.5.

End of Annex III.